Основы глубинного обучения

Лекция 5

Приёмы обучения нейросетей

Евгений Соколов

esokolov@hse.ru

НИУ ВШЭ, 2026

AdamW

- L2-регуляризация: добавляем к функции потерь $\frac{\lambda}{2}\|w\|^2$, в градиенте появляется слагаемое λw:

$$g_t = \nabla Q(w^{t-1}) + \lambda w^{t-1}$$

  - дальше обычный шаг Adam по g_t: штраф делится на $\sqrt{\hat{v}_t}$ вместе с градиентом
  - у параметров с большими градиентами регуляризация почти исчезает
- AdamW (decoupled weight decay): штраф вычитаем из весов отдельно, минуя нормировку:

$$w^t = w^{t-1} - \eta_t \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda w^{t-1} \right)$$

- Для SGD обе формы совпадают, для Adam — нет

https://arxiv.org/abs/1711.05101

Muon (2024–2025)

- Веса линейного слоя — матрица W, градиент по ним G — тоже матрица. Adam масштабирует каждый элемент G по отдельности и не видит, что это матрица
- Muon берёт градиент с инерцией M_t и заменяет его на ближайшую ортогональную матрицу: все сингулярные числа становятся равны 1

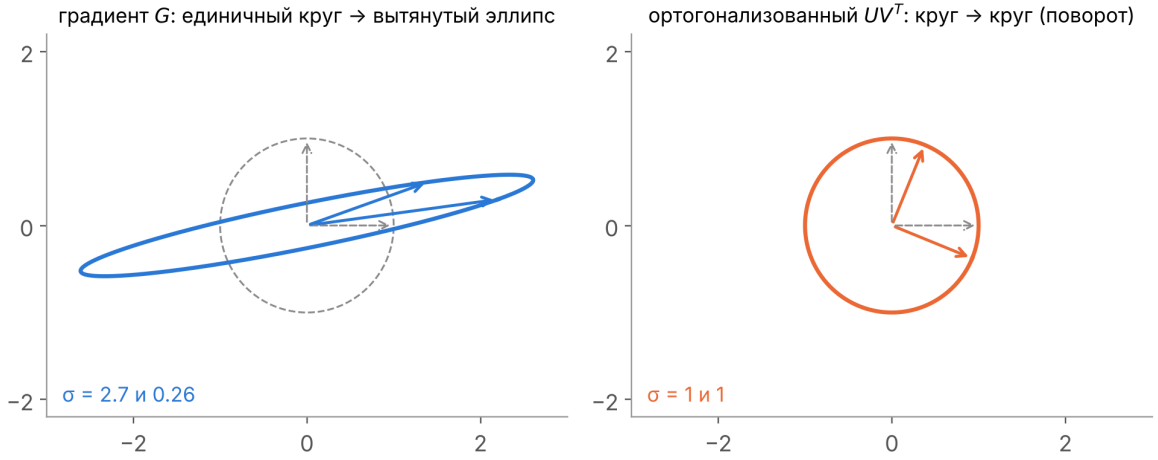
$$W^t = W^{t-1} - \eta_t \cdot UV^T, \qquad \text{где } M_t = U\Sigma V^T$$

  - смысл: обновление одинаково сильно двигает все направления, а не только несколько доминирующих
- Применяется к матрицам внутри сети; эмбеддинги, выходной слой и параметры нормализаций — по-прежнему AdamW
- Результат: до того же лосса примерно вдвое меньше вычислений, чем у AdamW (Moonshot, 2025); Kimi K2 (1 трлн параметров) обучена Muon

https://arxiv.org/abs/2502.16982

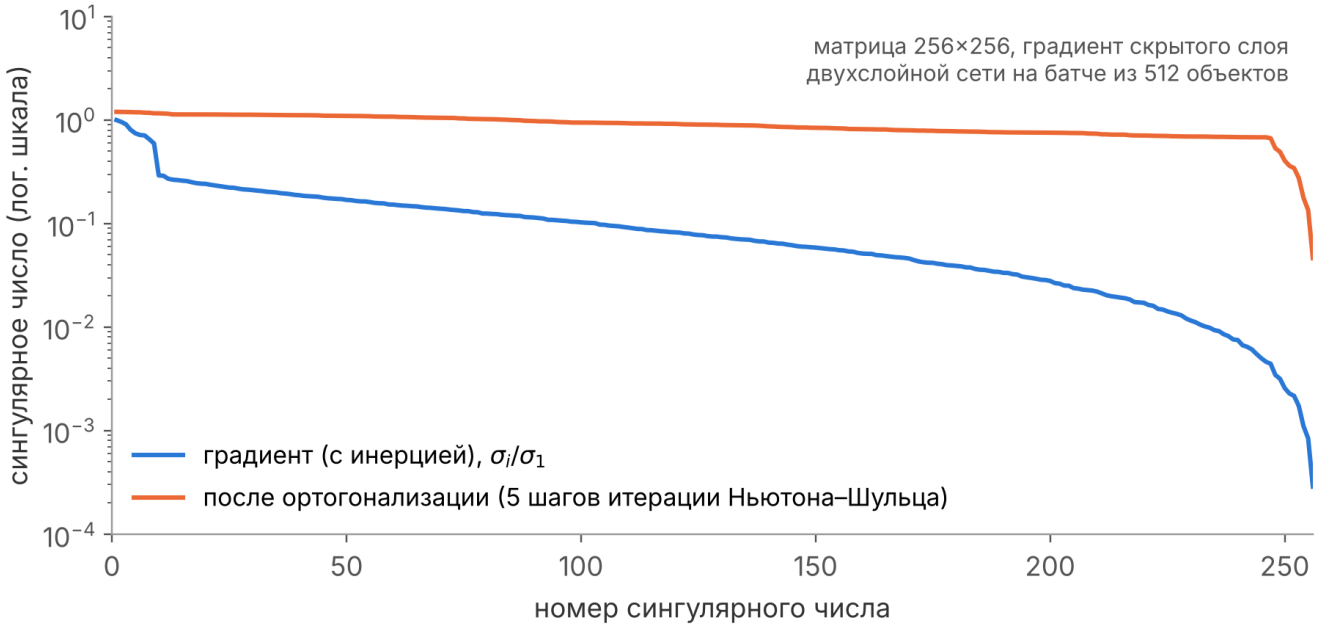
Ортогонализация

- Матрица обновления G — линейное отображение
  - единичный шар она растягивает в эллипсоид
  - полуоси — сингулярные числа σ_i
- Большое σ_i — направление, по которому шаг двигает веса сильно, маленькое — почти не двигает
- Ортогонализация оставляет направления (поворот UV^T) и выравнивает длины: все $\sigma_i = 1$
- Редкие направления получают такой же шаг, как доминирующие



Спектр матрицы производных

- Градиент матрицы весов почти низкоранговый: лишь несколько сингулярных чисел большие
  - на картинке 65% энергии обновления лежит в 10 направлениях из 256
- Adam масштабирует каждый элемент матрицы по отдельности и этого не видит
- Muon воздействует на матрицу целиком



https://kellerjordan.github.io/posts/muon/

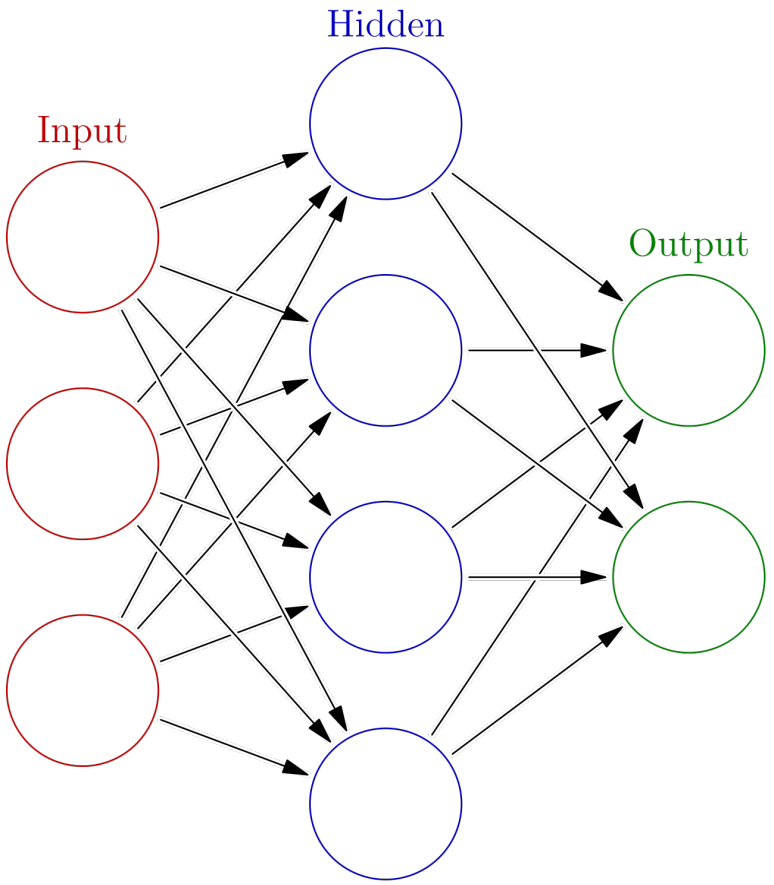
Dropout

Борьба с переобучением

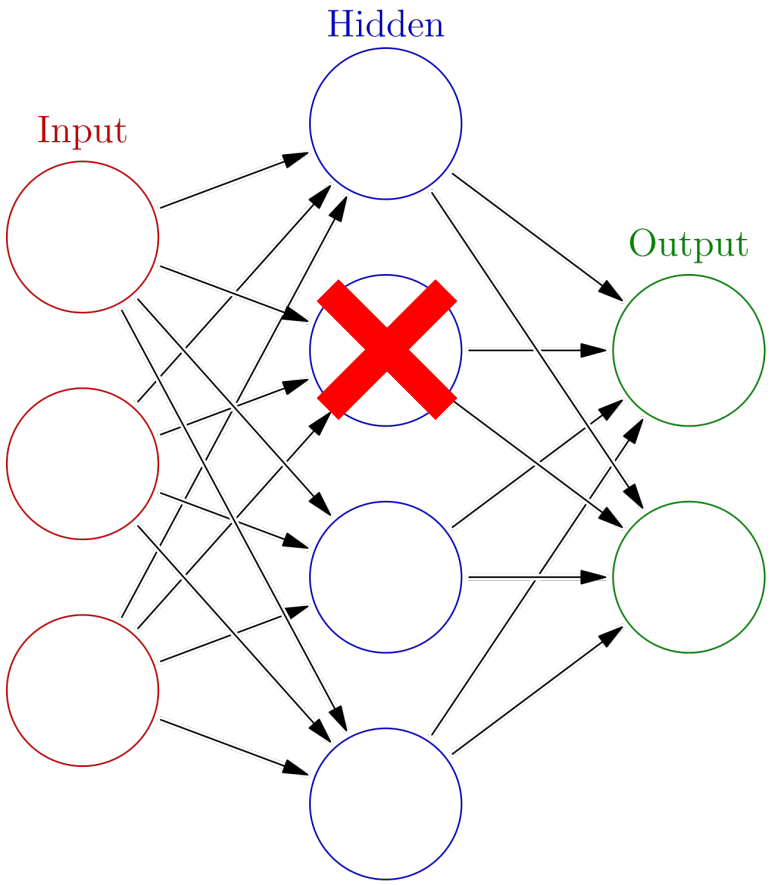
- Сокращение числа параметров (свёрточные слои помогают с этим)
- Регуляризация

- Можно как-то ещё мешать модели подгоняться под обучающую выборку

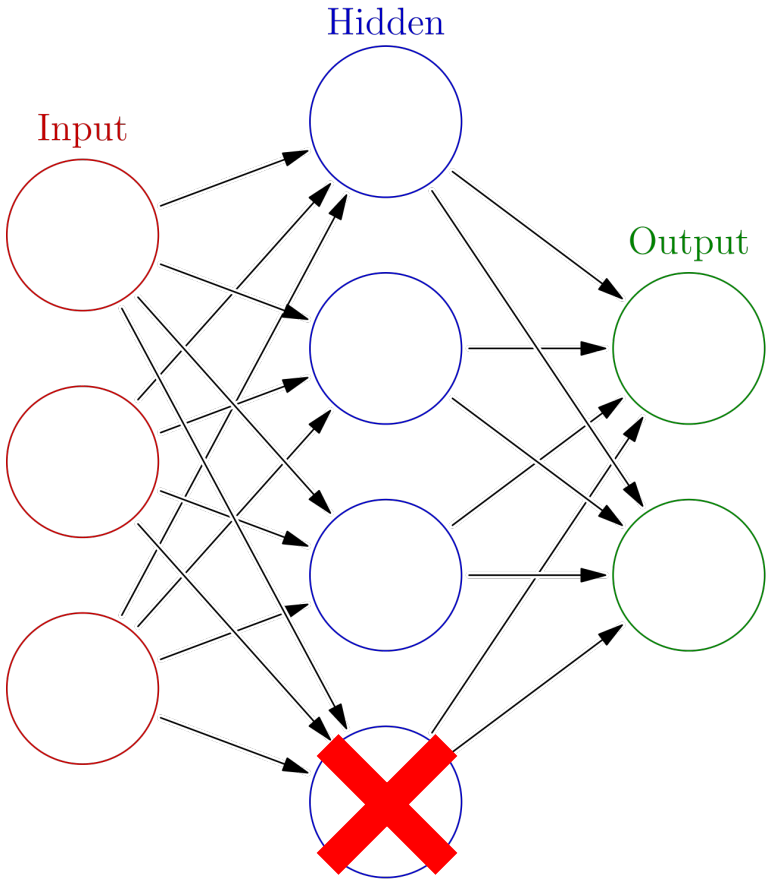
Dropout



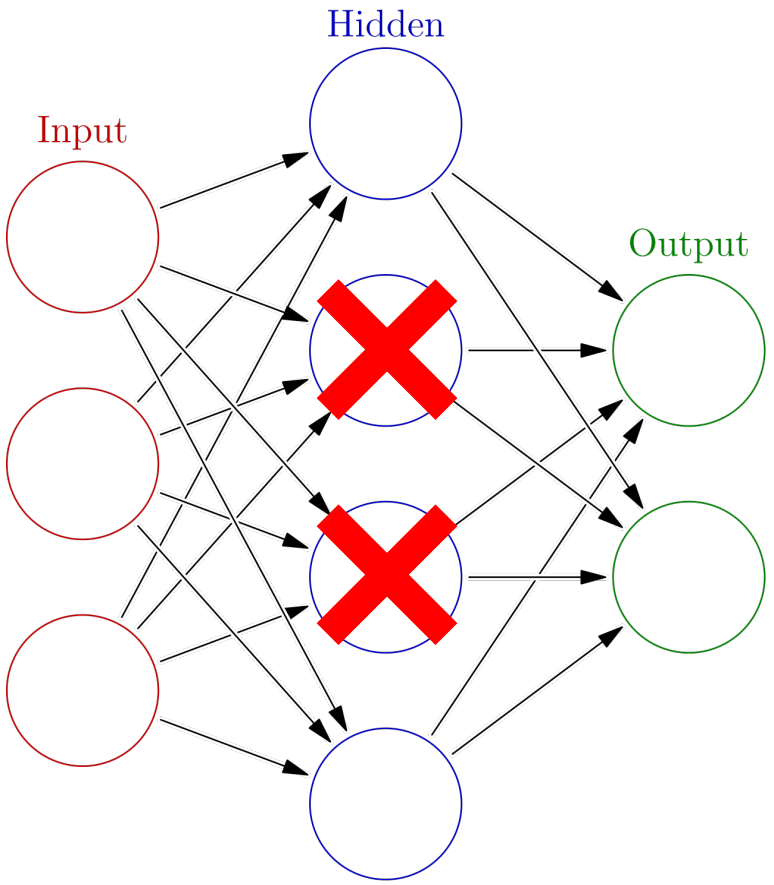
Dropout



Dropout



Dropout



Dropout

- Можно определить как слой $d(x)$
- Параметров нет, единственный гиперпараметр — p (вероятность удаления нейрона)
- На этапе обучения:

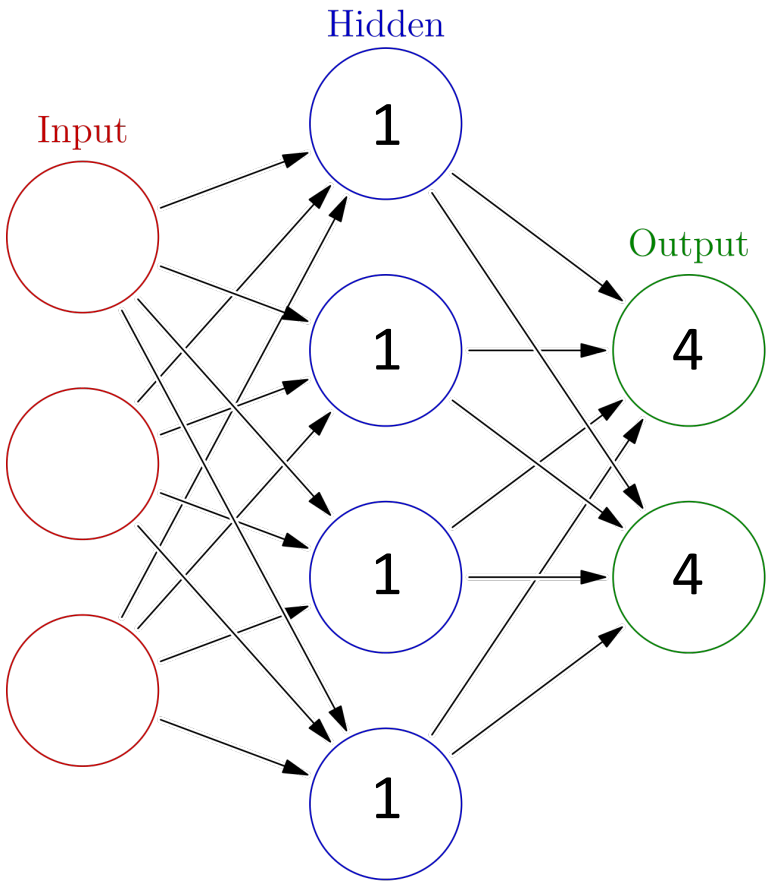
$$d(x) = \frac{1}{1-p} m \circ x$$

- m — вектор того же размера, что и x, элемент берутся из распределения $\mathrm{Ber}(1-p)$
- Деление на $(1-p)$ — для сохранения суммарного масштаба выходов

https://jmlr.org/papers/v15/srivastava14a.html

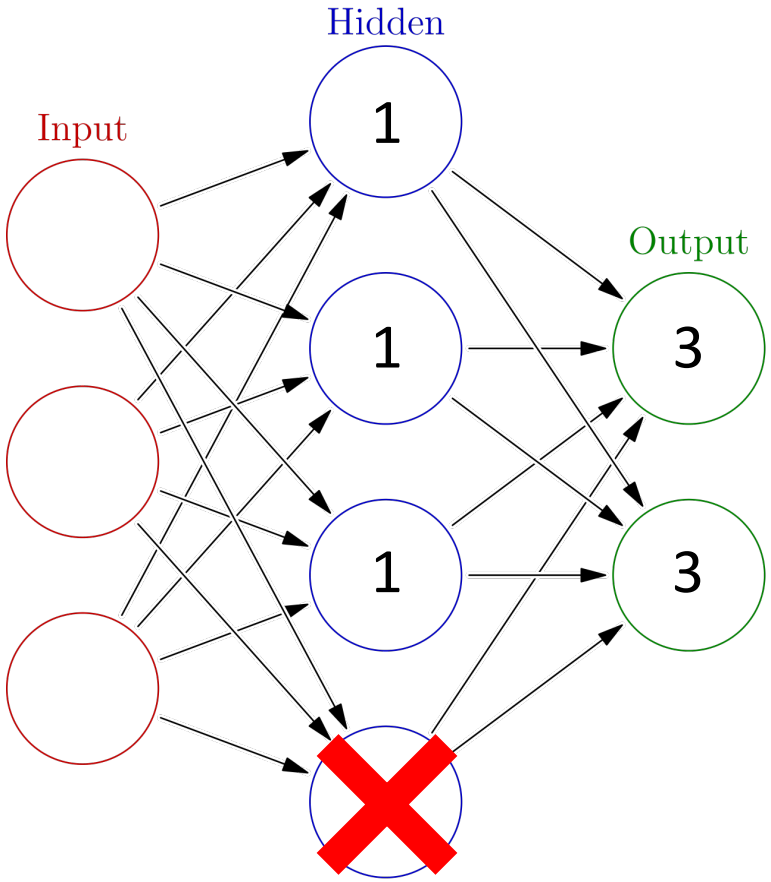
Dropout

Пусть все веса единичные



Dropout

Пусть все веса единичные



Надо компенсировать снижение масштаба суммы выходов!

Dropout

- На этапе обучения:

$$d(x) = \frac{1}{1-p} m \circ x$$

- На этапе применения:

$$d(x) = x$$

В оригинальной статье нет нормировки на этапе обучения, но есть домножение на $(1-p)$ на этапе применения

Вариант на слайде — inverted dropout (чуть меньше операций во время применения сети)

Dropout

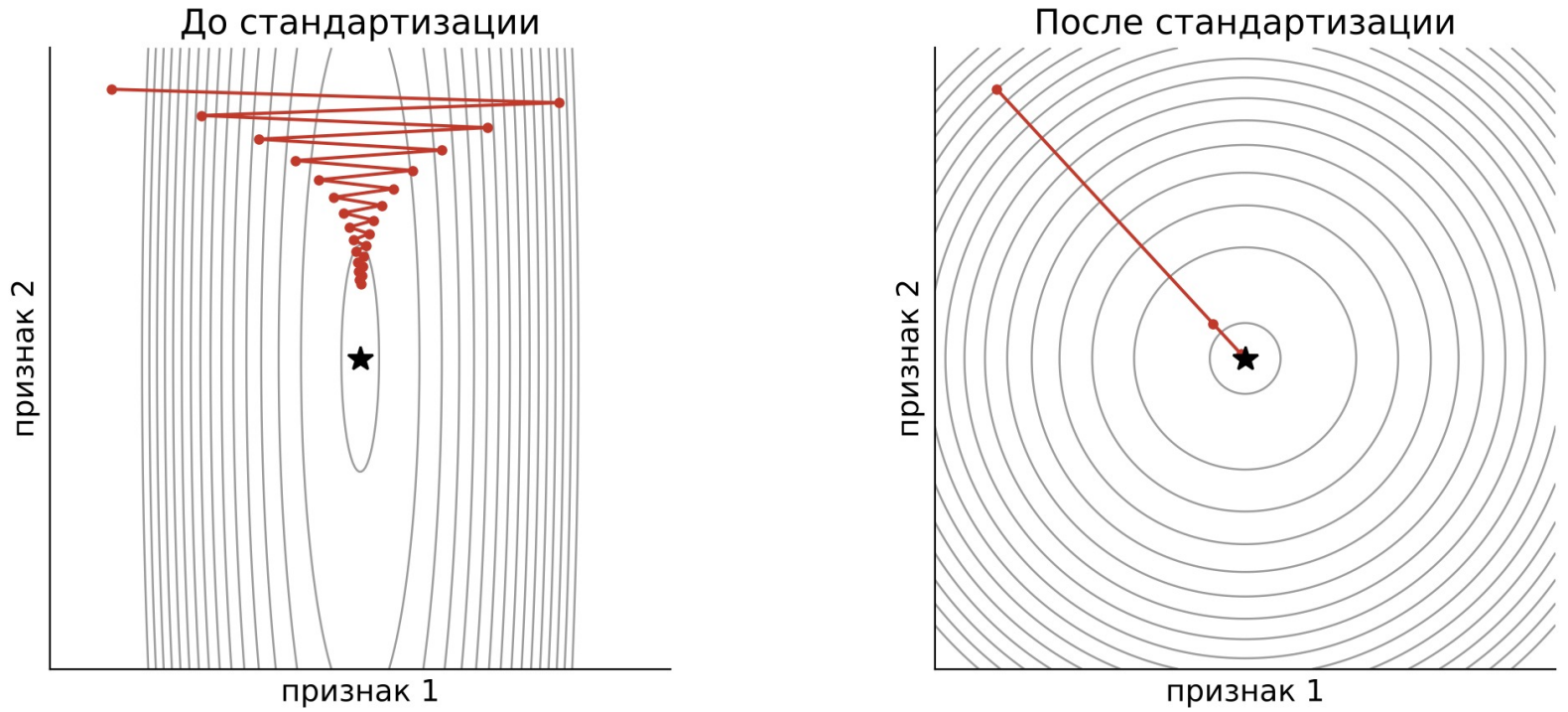
- Интерпретация: мы обучаем все возможные архитектуры нейросетей, которые получаются из исходной выбрасыванием отдельных нейронов
- У всех этих архитектур общие веса
- На этапе применения (почти) усредняем прогнозы всех этих архитектур

- Сейчас: в свёрточных сетях почти не используется, BatchNorm и аугментация работают лучше
- Применяется в полносвязных головах и в трансформерах ($p \approx 0.1$); в предобучении больших языковых моделей обычно не используется: данных больше, чем эпох, переобучаться не на чем

Нормализации

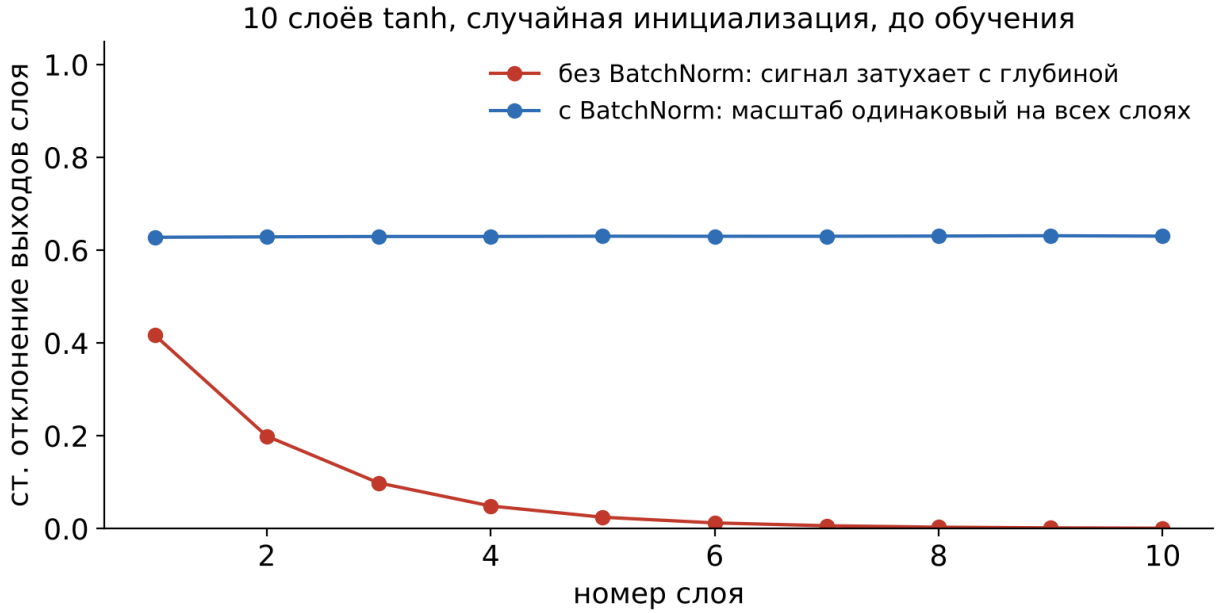
Масштабирование признаков

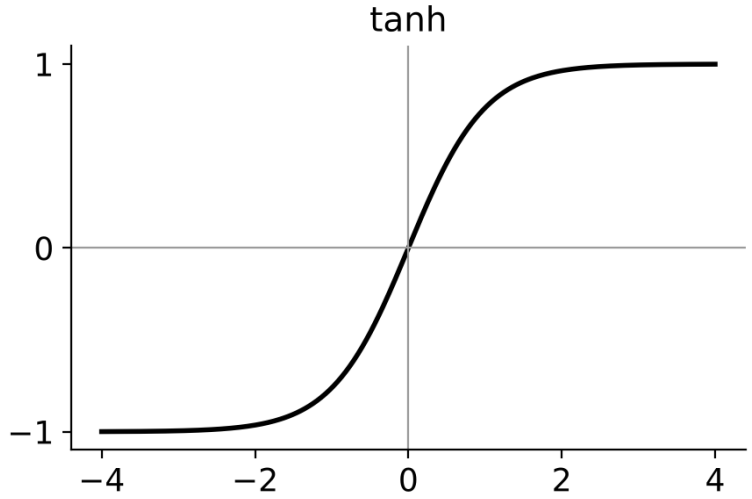
- В классическом ML признаки принято масштабировать
- Каждый слой нейросети — модель над выходами предыдущего
- Почему бы не масштабировать вход каждой модели?



Что происходит без масштабирования

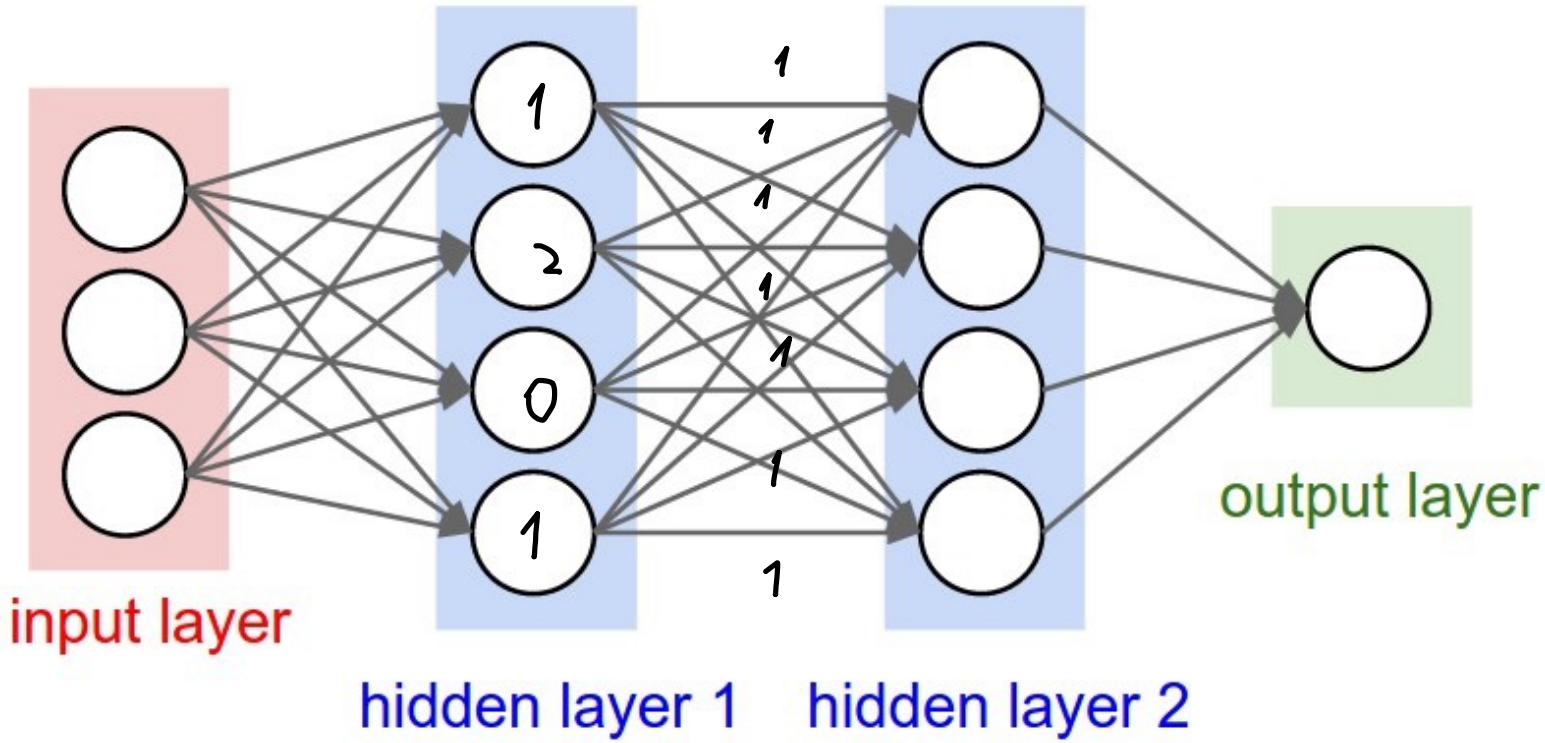
- Масштаб выходов меняется от слоя к слою: при неудачной инициализации сигнал затухает или насыщает нелинейность
- Градиент по весам слоя пропорционален его входам: $\frac{\partial L}{\partial W} = \delta \cdot x^T$. Где входы в 100 раз меньше, там при той же η шаг в 100 раз меньше
- Одна длина шага на все слои не подходит, пока входы у всех в разном масштабе





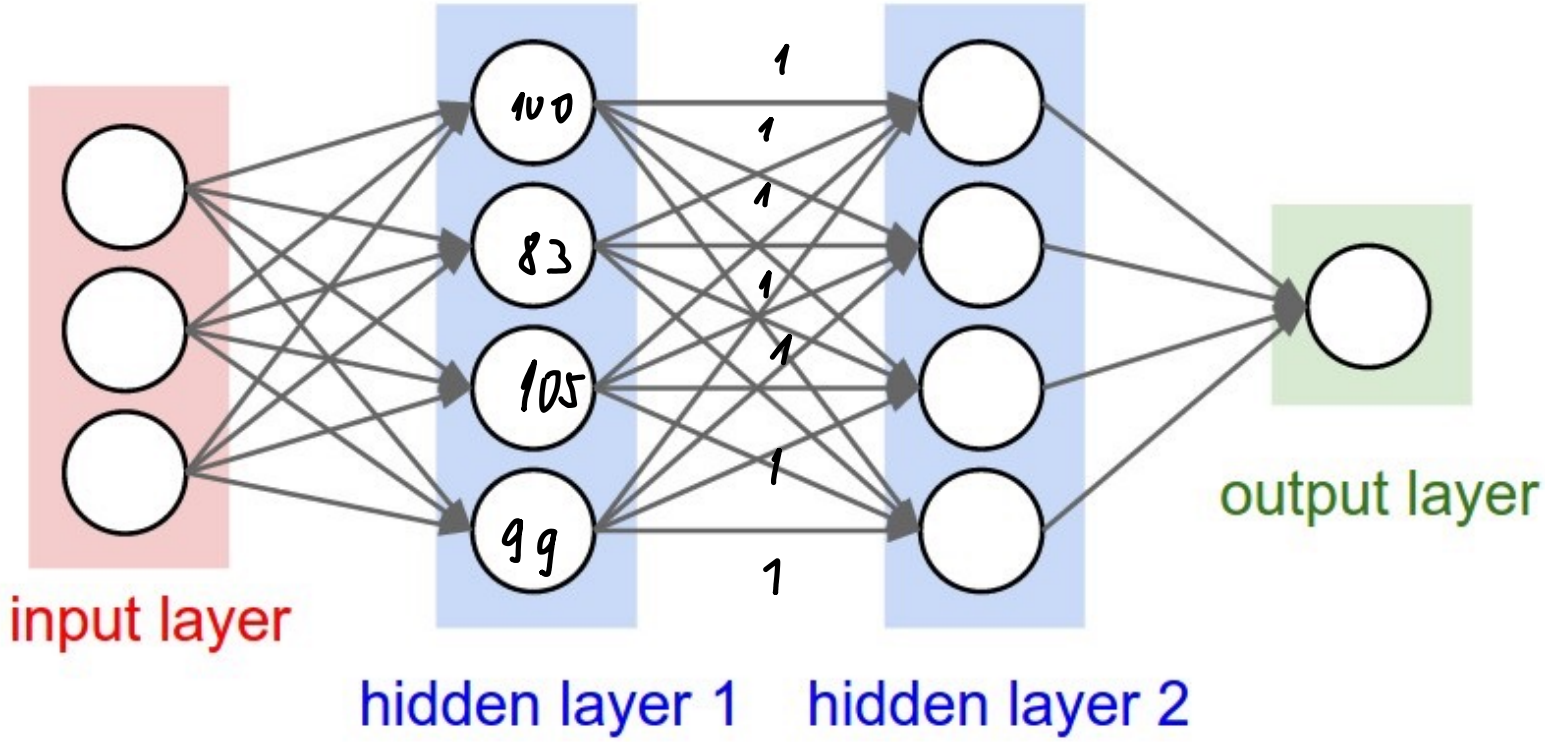
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Internal covariate shift



Internal covariate shift

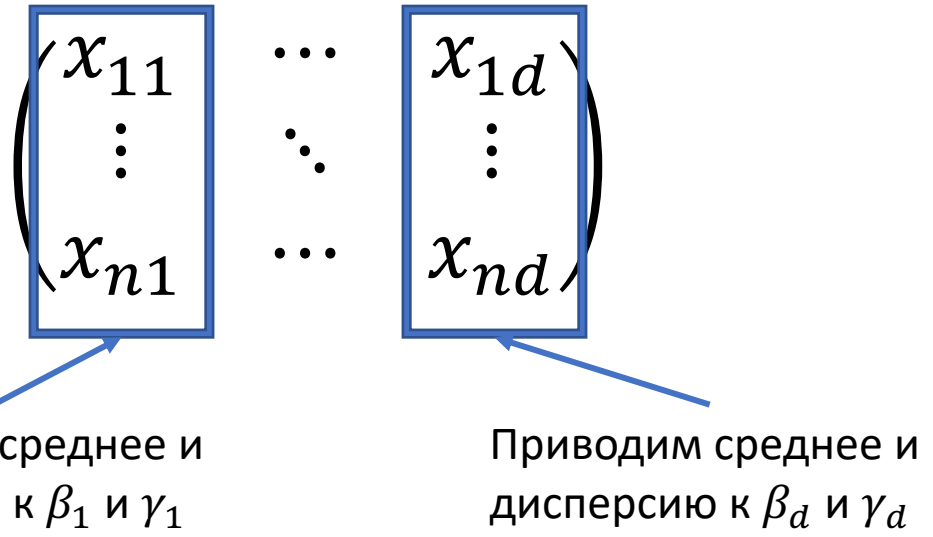
Допустим, веса первого слоя сильно поменялись после градиентного шага



Идея

- Входы слоя зависят от весов предыдущих слоёв, а они меняются на каждом шаге
- Пересчитывать по всей выборке на каждом шаге — дорого
- Выход: считать среднее и дисперсию по текущему батчу — он всё равно уже у нас в руках
- Авторы (Ioffe, Szegedy, 2015) назвали проблему internal covariate shift — но ещё обсудим, так ли это

Batch Normalization



- n — размер батча
- d — размерность входного вектора

Пример на числах

- Батч из 4 объектов, 3 признака
- Масштабируем каждый столбец со своими μ и σ

Батч на входе слоя

	рост	стаж	возраст
объект 1	180	2	30
объект 2	165	0.5	45
объект 3	172	8	22
объект 4	158	1.2	61

μ = 2.92

σ = 2.98

$(x - \mu) / \sigma$
по столбцам



После BatchNorm

рост	стаж	возраст
+1.38	-0.31	-0.64
-0.46	-0.81	+0.37
+0.40	+1.70	-1.17
-1.32	-0.58	+1.44

каждый столбец: μ = 0, σ = 1

Batch Normalization

- Реализуется как отдельный слой
- Вычисляется для текущего батча
- Оценим среднее и дисперсию каждой компоненты входного вектора:

$$\mu_B = \frac{1}{n}\sum_{j=1}^{n} x_{B,j}$$

покоординатно

$$\sigma_B^2 = \frac{1}{n}\sum_{j=1}^{n}\left(x_{B,j} - \mu_B\right)^2$$

$x_{B,j}$ — j-й объект в батче B

Batch Normalization

- Отмасштабируем все выходы:

$$\tilde{x}_{B,j} = \frac{x_{B,j} - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

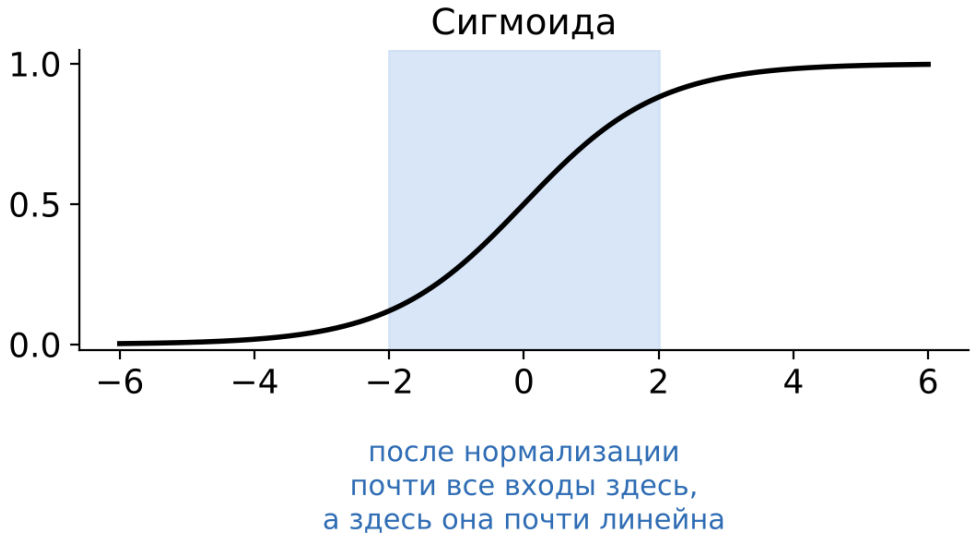
- Зададим нужные нам среднее и дисперсию:

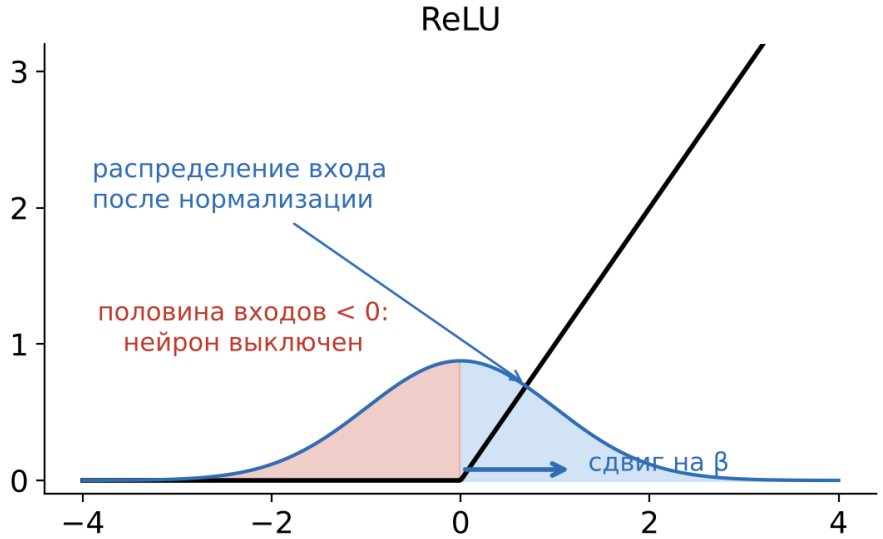
$$z_{B,j} = \gamma \circ \tilde{x}_{B,j} + \beta$$

обучаемые параметры (векторы, размерность равна размерности входных векторов)

Зачем γ и β

- Без них выход каждого нейрона имеет среднее 0 и дисперсию 1 — а это не всегда то, что нужно следующему слою
  - сигмоида: входы почти всегда в [−2, 2], где она почти линейна — нелинейность пропадает
  - ReLU: среднее 0 → половина входов отрицательна, для каждого объекта половина нейронов выключена; β сдвигает порог, а γ для ReLU почти не важна: $\mathrm{ReLU}(\gamma x) = \gamma \cdot \mathrm{ReLU}(x)$
- γ и β обучаемые: сеть сама выбирает масштаб и сдвиг каждого выхода; может даже отменить нормализацию ($\gamma = \sigma, \beta = \mu$)
- Среднее и дисперсия стали явными параметрами





Batch Normalization

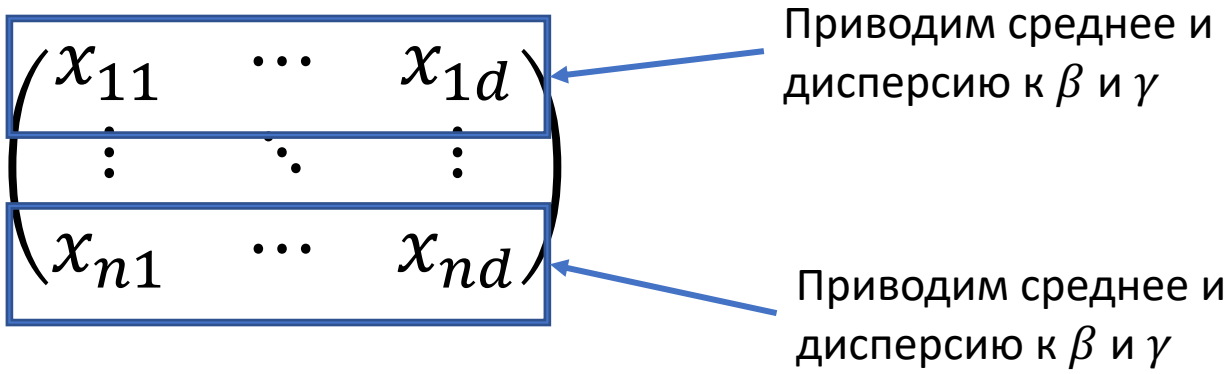
Важно: после BatchNorm среднее и дисперсия каждого выхода зависят только от параметров нормализации, но не от параметров прошлых слоёв!

Batch Normalization

Во время применения нейронной сети:

- Те же самые формулы, но вместо μ_B и σ_B^2 используем их средние значения по всем батчам

Layer Normalization



- n — размер батча
- d — размерность входного вектора
- Нормируем внутри одного объекта, по его признакам: от размера батча не зависит, на обучении и применении работает одинаково

https://arxiv.org/pdf/1607.06450.pdf

Layer Normalization

- Нормализуем распределение «признаков» одного объекта

$$\mu_i = \frac{1}{d}\sum_{j=1}^{d} x_{ij}$$

$$\sigma_i^2 = \frac{1}{d}\sum_{j=1}^{d}\left(x_{ij} - \mu\right)^2$$

x_{ij} — j-й признак i-го объекта

https://arxiv.org/pdf/1607.06450.pdf

Layer Normalization

- Отмасштабируем все выходы:

$$\tilde{x}_{ij} = \frac{x_{ij} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}$$

- Зададим нужные нам среднее и дисперсию:

$$z_{ij} = \gamma \circ \tilde{x}_{ij} + \beta$$

обучаемые параметры — векторы размерности d: свои gain и bias у каждого нейрона

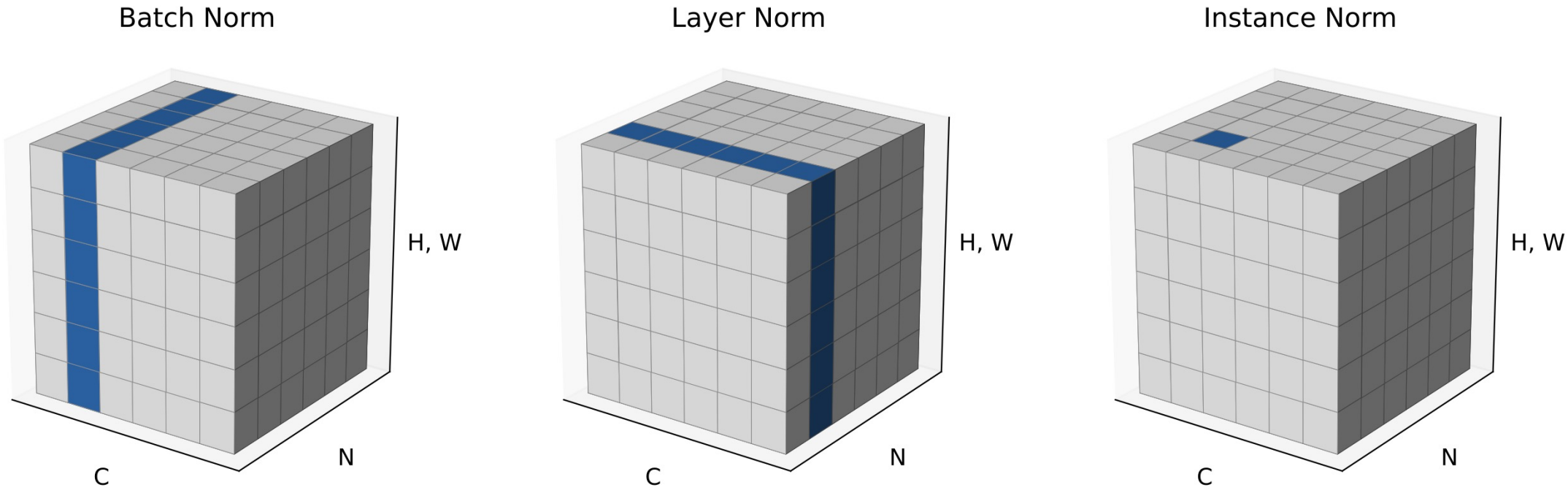
https://arxiv.org/pdf/1607.06450.pdf

BatchNorm для свёрточных слоёв

- Вход — тензор $N \times C \times H \times W$
  - N — объектов в батче, C — каналов, H и W — высота и ширина карты
- Статистики считаются на канал: усредняем по объектам и по всем позициям, то есть по осям N, H, W
  - на каждый канал — одно μ, одно σ^2, одна пара γ и β; итого $2C$ обучаемых параметров
- Почему так: свёртка одна и та же для всех позиций карты, значит и масштабирование канала должна быть одинаковой во всех позициях

По каким осям усредняем

- Batch Norm: по объектам и по позициям — одно μ и σ на канал
- Layer Norm: по каналам и позициям — одно μ и σ на объект
- Instance Norm: только по позициям — на объект и канал

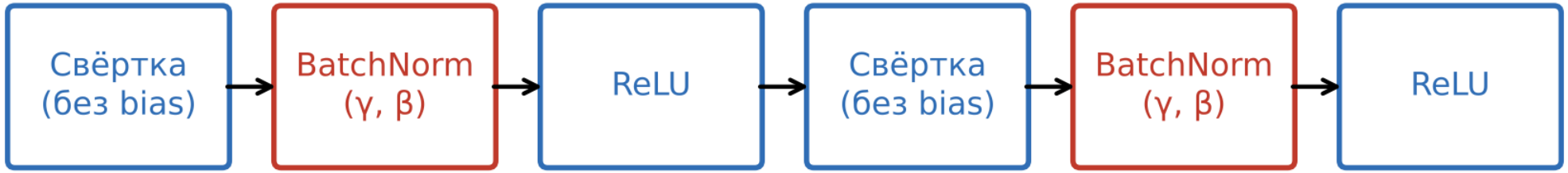


Синие ячейки нормируются одним и тем же μ и σ, посчитанными по этим же ячейкам

Batch Normalization

- Обычно вставляется между полносвязным/свёрточным слоём и нелинейностью
- Позволяет увеличить длину шага в градиентном спуске

- Не факт, что действительно устраняет covariate shift



bias перед BatchNorm бесполезен: вычитание среднего его обнуляет, за сдвиг отвечает β

В чём польза от BatchNorm?

How Does Batch Normalization Help Optimization?

Shibani Santurkar* MIT shibani@mit.edu

Dimitris Tsipras* MIT tsipras@mit.edu

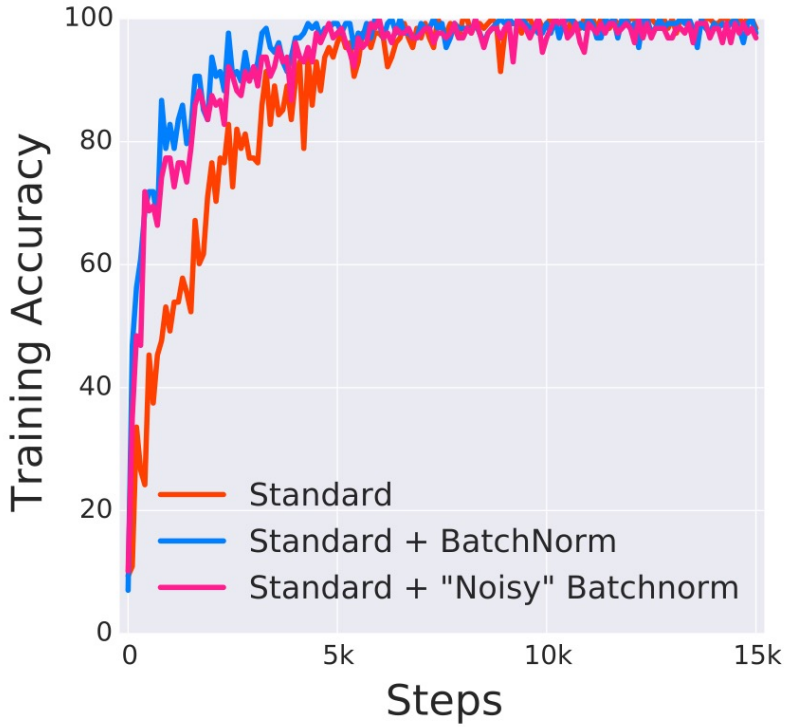
Andrew Ilyas* MIT ailyas@mit.edu

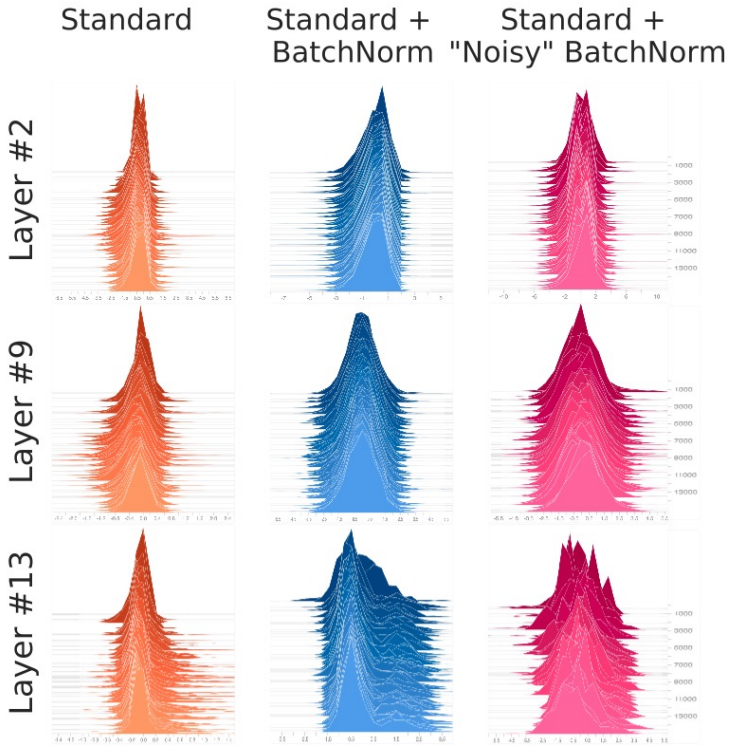
Aleksander Mądry MIT madry@mit.edu

https://arxiv.org/pdf/1805.11604.pdf

В чём польза от BatchNorm?

- Добавим шум после нормализации — хуже не становится!

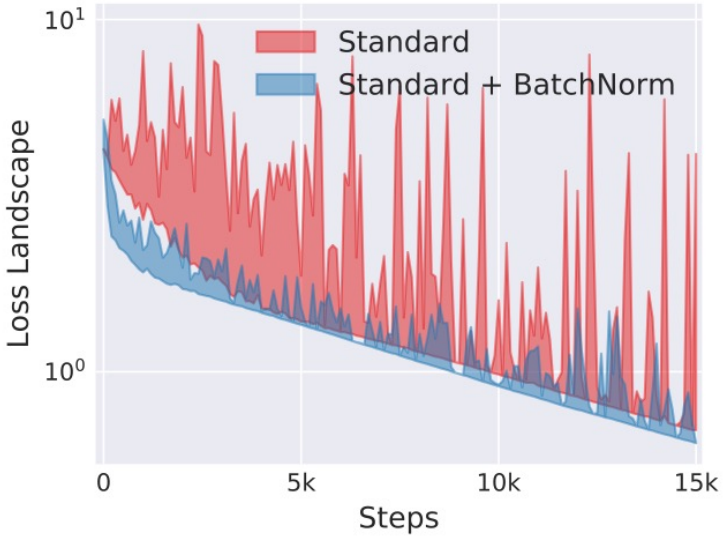




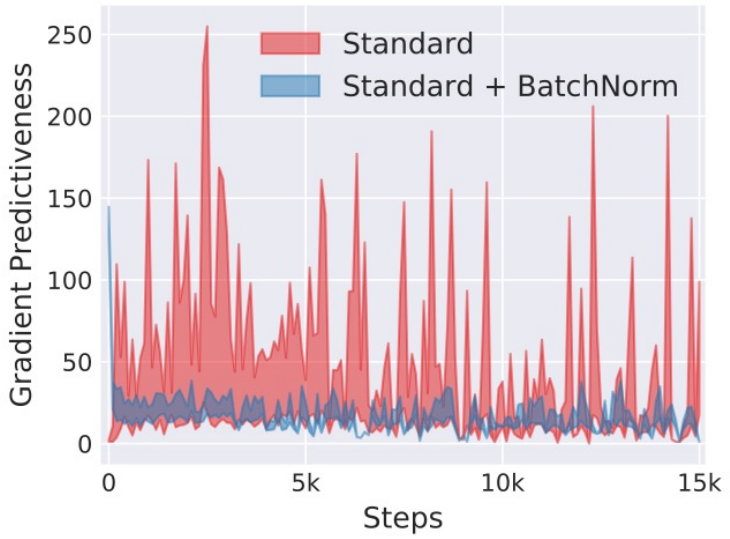
https://arxiv.org/pdf/1805.11604.pdf

В чём польза от BatchNorm?

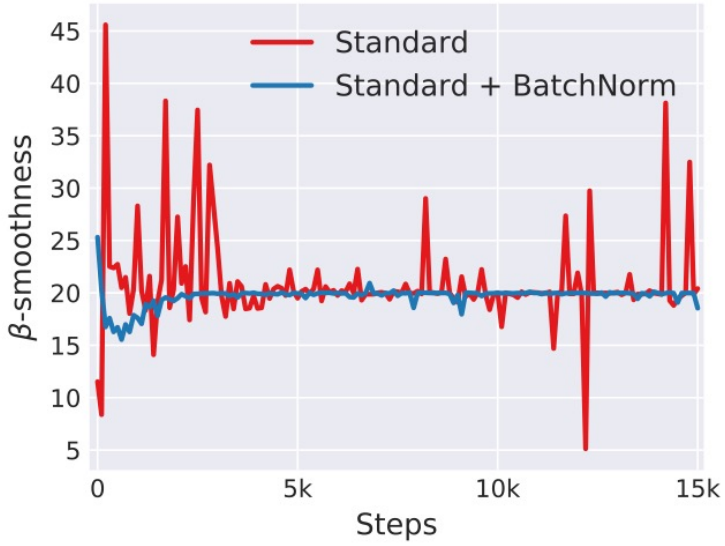
- Функционал ошибки становится более гладким!



(a) loss landscape



(b) gradient predictiveness



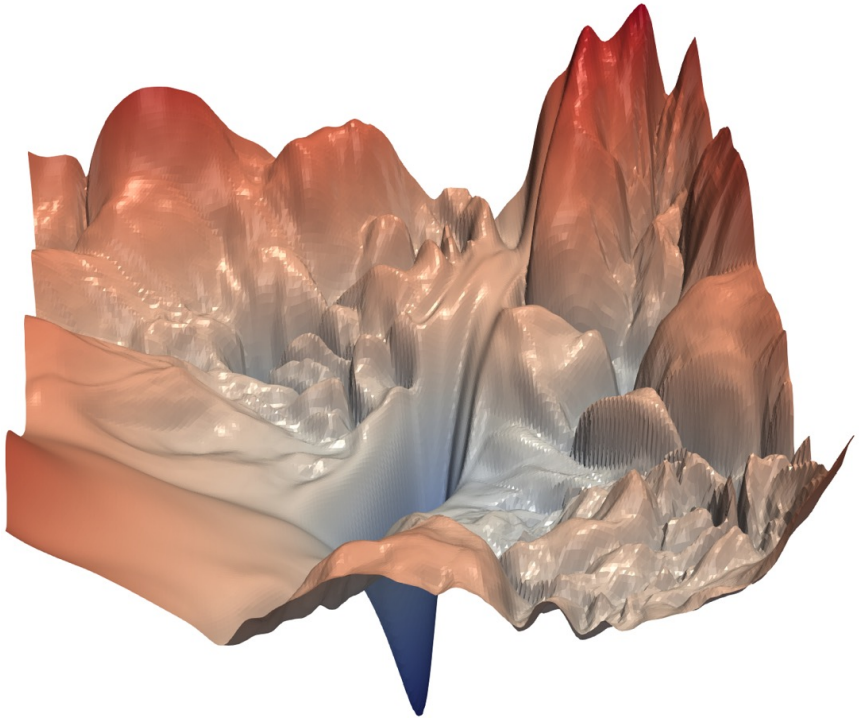
(c) "effective" β-smoothness

https://arxiv.org/pdf/1805.11604.pdf

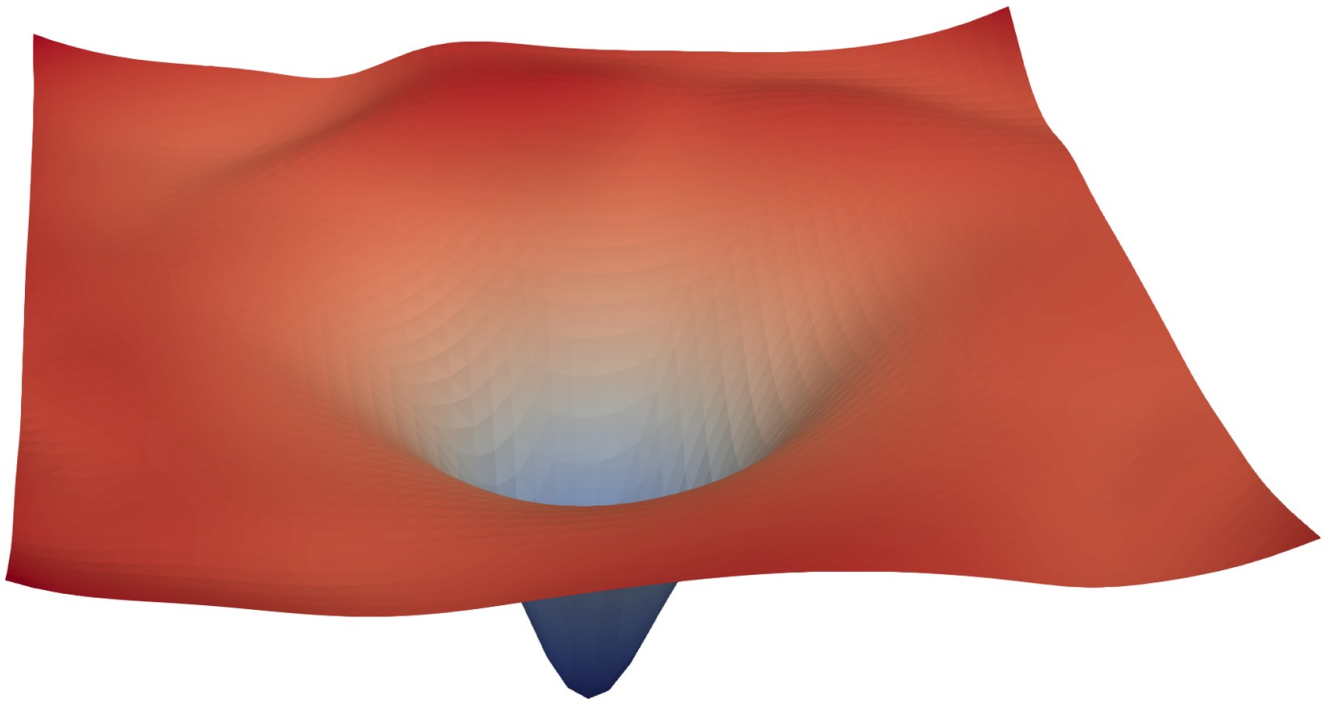
Ландшафт лосса и skip connections

- ResNet-56 на CIFAR-10, срез функционала ошибки по двум случайным направлениям в пространстве весов
- Без skip connections — хаотичный рельеф, со skip connections — почти выпуклая чаша вокруг минимума
  - Li, Xu, Taylor, Studer, Goldstein. Visualizing the Loss Landscape of Neural Nets, NeurIPS 2018 — arxiv.org/abs/1712.09913

ResNet-56 без skip connections

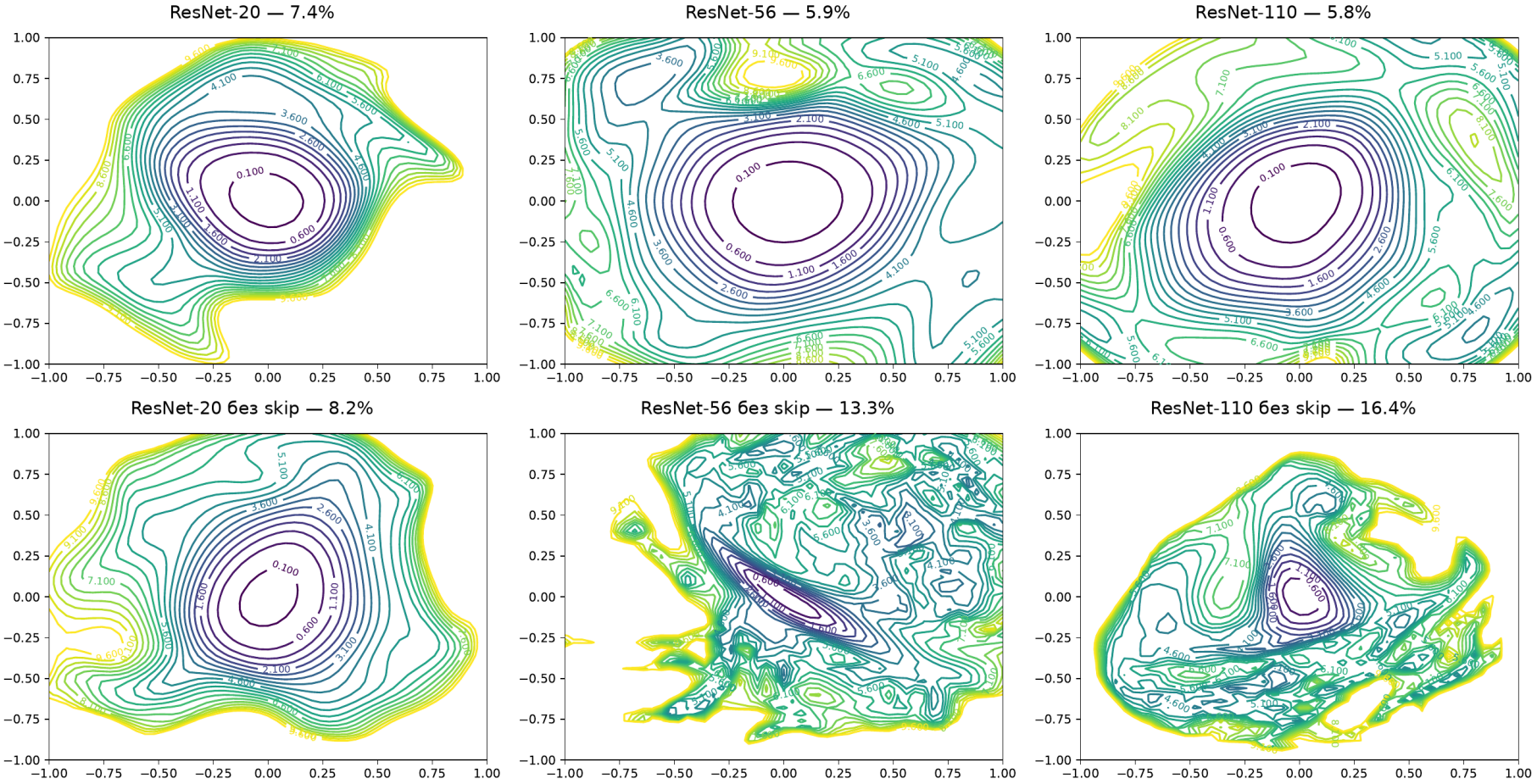


ResNet-56 со skip connections



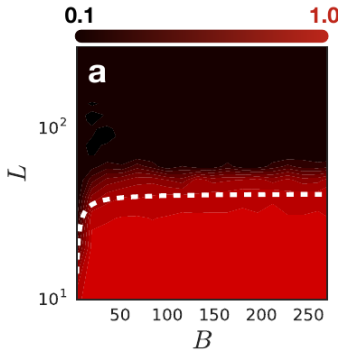
Глубина против skip connections

- Те же срезы для ResNet-20 / 56 / 110; в подписи — ошибка на тесте
- Чем ровнее картинка, тем лучше обобщение
  - Li et al., 2018, Figure 5 (срезы с filter normalization)



BatchNorm без skip connections

- В сети без skip connections BatchNorm сам вызывает экспоненциальный рост градиентов с глубиной
- В обычной сети сигнал и градиент на каждом слое умножаются на множитель, зависящий от дисперсии весов — инициализация (He: дисперсия 2/n) выравнивает оба прохода
- BatchNorm приводит активации к единичной дисперсии при любых весах: прямой проход от инициализации больше не зависит, и выровнять обратный нечем. Множитель на слой задаёт только нелинейность: для ReLU $\frac{\pi}{\pi-1} \approx 1.47$, для линейной — 1
- Итог: без residual-связей сеть с BatchNorm обучается примерно до 50 слоёв ($1.47^{50} \approx 2 \cdot 10^{8}$), глубже — нет
- На картинке: по вертикали глубина L, по горизонтали размер батча B, цвет — точность на MNIST после 10 эпох (красное — обучилась, чёрное — нет), белый пунктир — предсказание теории
- Skip connections дают путь с множителем 1: взрыв по BN-ветке складывается, а не умножается



https://arxiv.org/abs/1902.08129

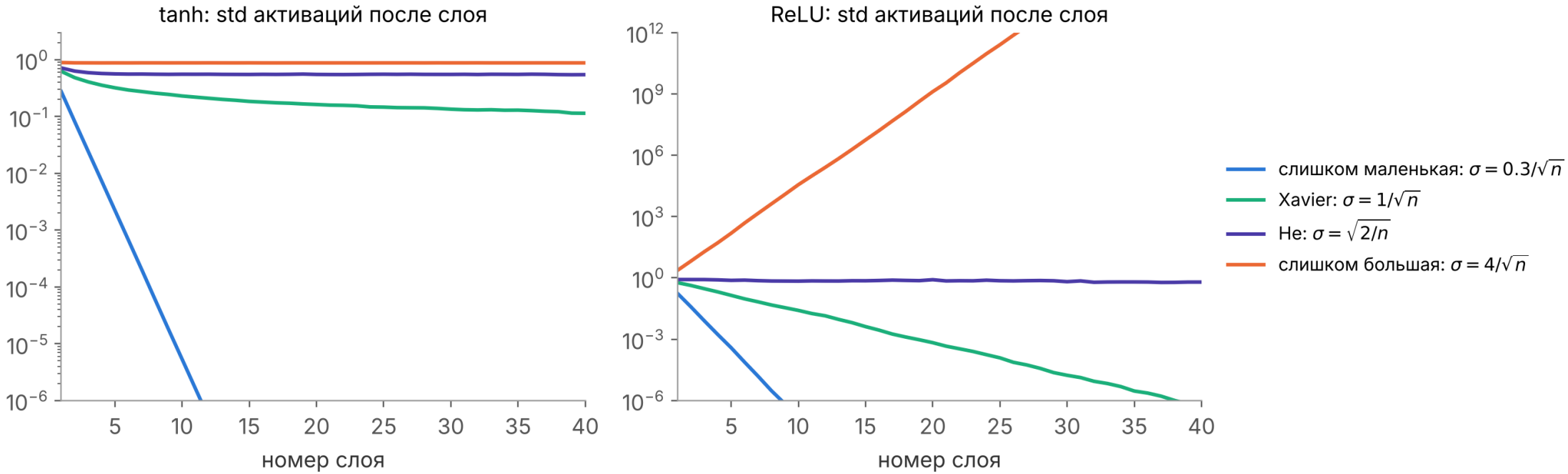
Проблемы BatchNorm

- Зависимость от размера батча: при батче 1–8 статистики шумные, при батче 1 дисперсия нулевая — нормировать нечего
- Разное поведение на обучении и применении: `model.train()` / `model.eval()`
  - забытый `eval()` — предсказания зависят от соседей по батчу
- Объекты в батче перестают быть независимыми — статистики связывают их
- В трансформерах длина последовательности переменная — батчевые статистики неудобны
- Выход: нормировать не по батчу, а внутри одного объекта → Layer Normalization

Инициализация

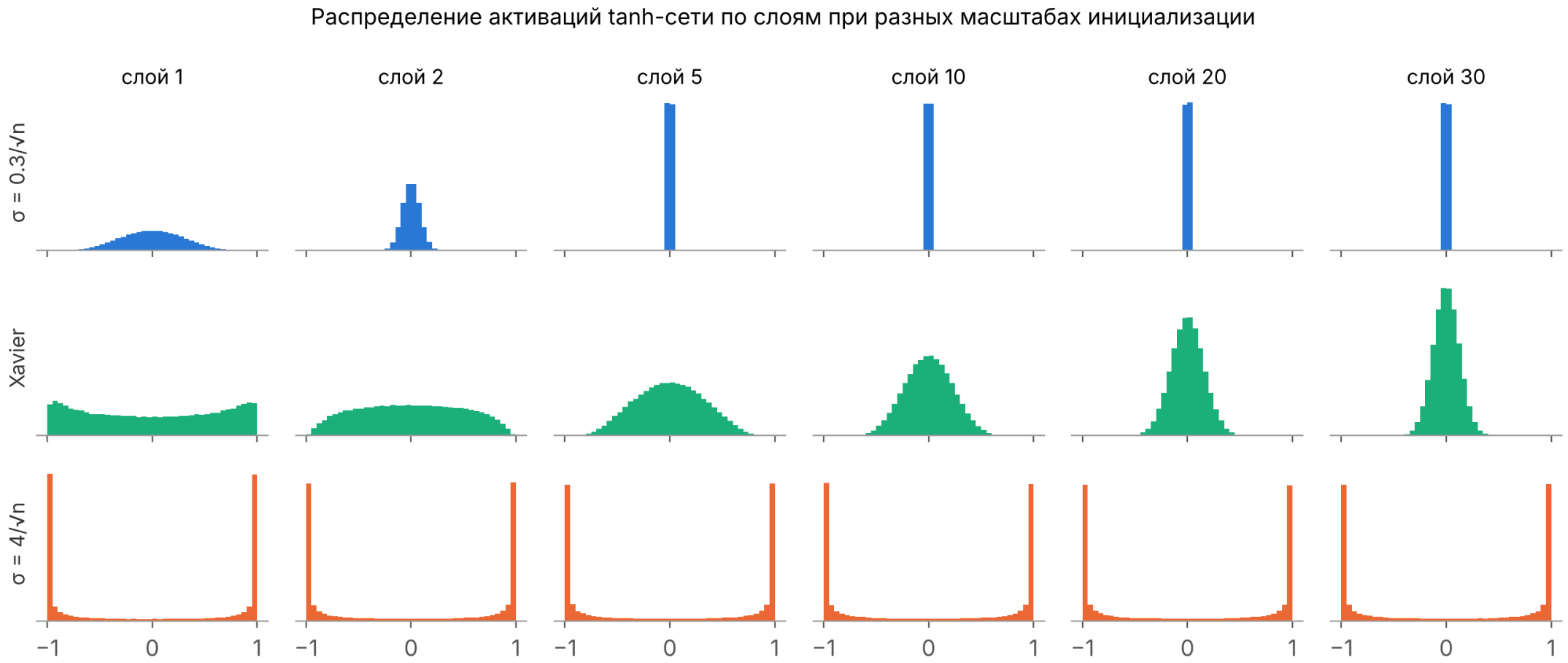
Зачем думать об инициализации

- Нулями или одним числом нельзя: все нейроны слоя получают одинаковые градиенты и навсегда остаются копиями друг друга
- Случайно тоже плохо
  - Маленькие веса — сигнал затухает с глубиной
  - Большие — tanh и сигмоида насыщаются, а с ReLU сигнал взрывается
- У обратного прохода те же проблемы
  - Градиент умножается на матрицу каждого слоя



Как это выглядит

- 30-слойная tanh-сеть ширины 256 на случайном входе: распределение активаций по слоям



Инициализация Xavier (2010)

- Линейный нейрон с n_{in} входами, веса и входы независимы и центрированы:

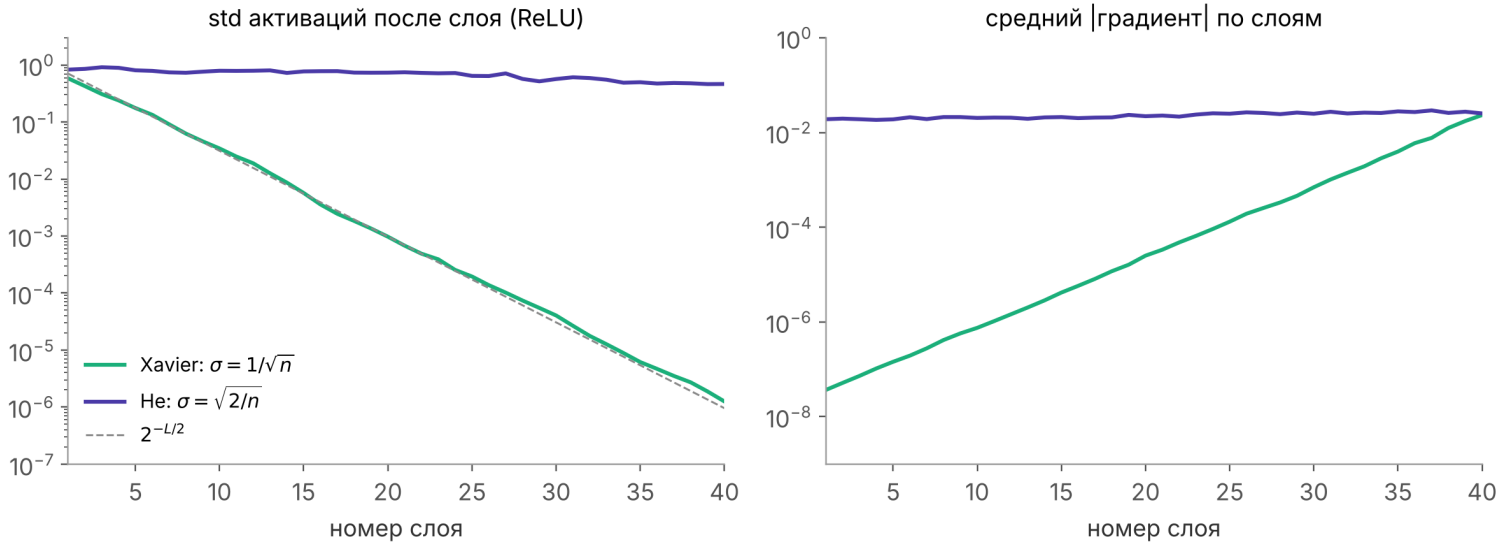
$$z = \sum_{i=1}^{n_{in}} w_i x_i\,, \qquad \mathbb{D}z = n_{in} \cdot \mathbb{D}w \cdot \mathbb{D}x$$

- Хотим сохранять масштаб сигнала, $\mathbb{D}z = \mathbb{D}x$: $\mathbb{D}w = \frac{1}{n_{in}}$; для градиентов то же с n_{out}
- Компромисс Glorot & Bengio: $\mathbb{D}w = \frac{2}{n_{in}+n_{out}}$
- Равномерный вариант: $U(-a, a)$, $a = \sqrt{\frac{6}{n_{in}+n_{out}}}$; для свёртки $n_{in} = k \cdot k \cdot C_{in}$
- Вывод сделан для линейной активации; для tanh около нуля это почти так же

https://proceedings.mlr.press/v9/glorot10a.html

Инициализация He (2015) для ReLU

- Слой с ReLU: $z = \sum_{i=1}^{n_{in}} w_i x_i$, $h = \mathrm{ReLU}(z) = \max(0, z)$ — вход следующего слоя
- z симметричен относительно нуля, половина значений обнуляется: $\mathbb{E}h^2 = \frac{1}{2}\mathbb{E}z^2 = \frac{1}{2}\mathbb{D}z$
- Следующий слой: $\mathbb{D}z' = n_{in} \cdot \mathbb{D}w' \cdot \mathbb{E}h^2 = n_{in} \cdot \mathbb{D}w' \cdot \frac{\mathbb{D}z}{2}$ → с Xavier масштаб падает вдвое на каждом слое, через 30 слоёв — в 2^{15} раз
- Поправка: $\mathbb{D}w = \frac{2}{n_{in}}$. Конкретно: $w \sim \mathrm{N}\left(0, \frac{2}{n_{in}}\right)$ или $\mathrm{U}(-a, a)$, $a = \sqrt{\frac{6}{n_{in}}}$; для свёртки $n_{in} = k \cdot k \cdot C_{in}$
- Из статьи: 22-слойная ReLU-сеть учится с обеими инициализациями, 30-слойная с Xavier не учится вовсе (градиенты затухают)



https://arxiv.org/abs/1502.01852

Инициализация на практике

- Принцип: масштаб сигнала и градиента не должен меняться от слоя к слою, формула зависит от нелинейности
- BatchNorm/LayerNorm сами приводят масштаб в порядок на каждом слое — поэтому вопрос стал менее острым
- Что делают сейчас:
  - берут значения по умолчанию из библиотеки (PyTorch: равномерная на $\pm 1/\sqrt{n_{in}}$ для Linear и Conv) и обычно не трогают — вместе с нормализацией этого хватает
  - в очень глубоких сетях следят, чтобы масштаб не рос с глубиной: последний слой каждого блока инициализируют маленьким или нулём, чтобы на старте сеть вела себя как мелкая
  - в больших моделях масштаб инициализации и длину шага согласуют с шириной сети, чтобы гиперпараметры, подобранные на маленькой модели, переносились на большую (µP, Yang et al., 2022)
  - чаще всего стартуют вовсе не со случайных весов, а с предобученной модели

https://arxiv.org/abs/2203.03466

Аугментация

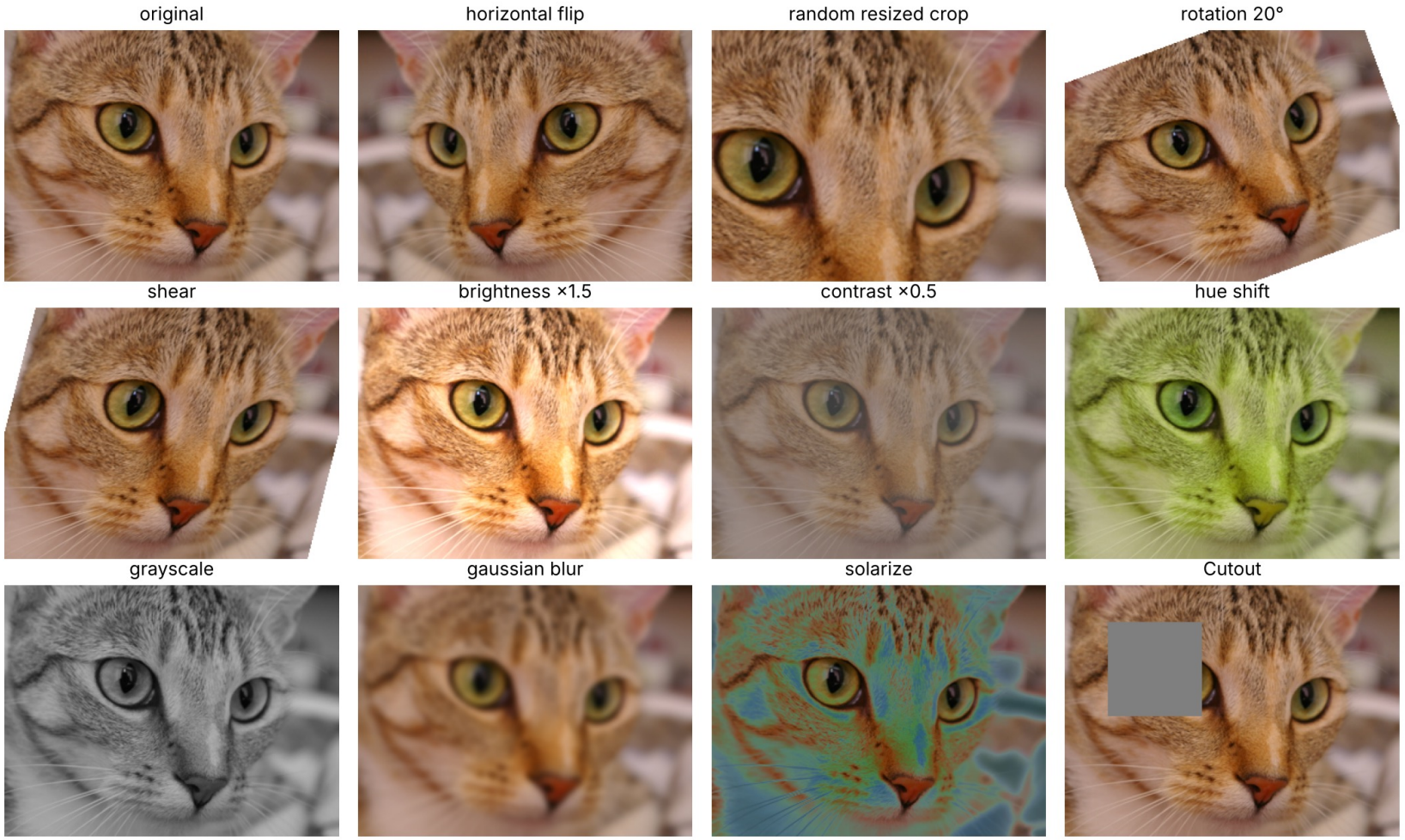
Аугментация: идея

- Мы заранее знаем, к чему модель должна быть устойчива: сдвиг, отражение, освещение, небольшой поворот
- Вряд ли стоит надеяться, что модель выучит это сама
- Синтезируем новые примеры небольшими возмущениями имеющихся — получим бесплатное расширение выборки
- По сути регуляризация: вшиваем в данные априорное знание об инвариантностях

- На практике — случайные преобразования к каждому батчу на лету
  - одна и та же картинка в каждой эпохе немного другая
- На валидации и тесте — без аугментаций (кроме test-time augmentation)

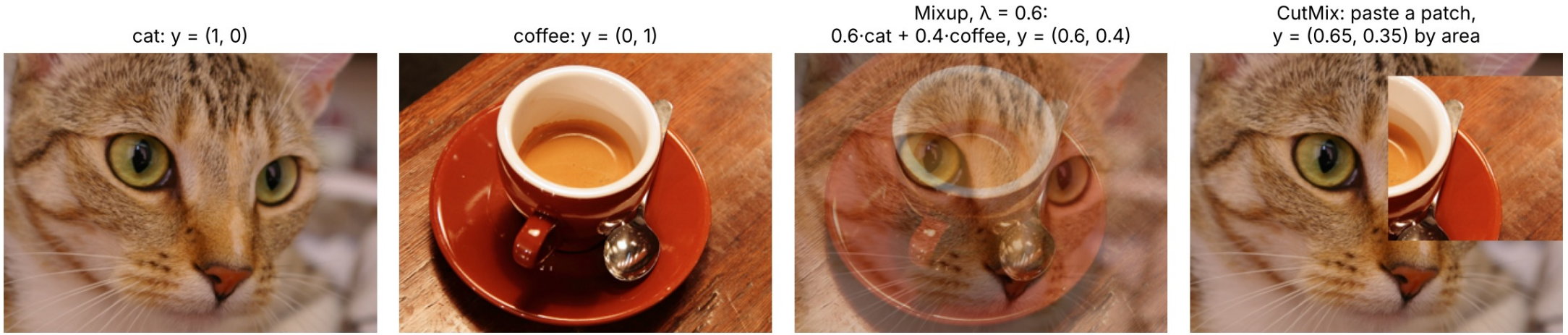
Примеры аугментаций

- Классический подход на ImageNet (ResNet, 2015) — случайный кроп с изменением масштаба и горизонтальное отражение
- Сегодня смешивают большее число аугментаций



Выбрасывание и смешивание

- Cutout / Random Erasing (2017): закрашиваем случайный прямоугольник — сеть не может полагаться на одну деталь
- Mixup (2018): $x = \lambda x_i + (1 - \lambda)x_j$, метки смешиваются так же; λ ~ Beta(α, α)
- CutMix (2019): вставляем кусок одной картинки в другую, метки — пропорционально площади



https://arxiv.org/abs/1710.09412 · https://arxiv.org/abs/1905.04899

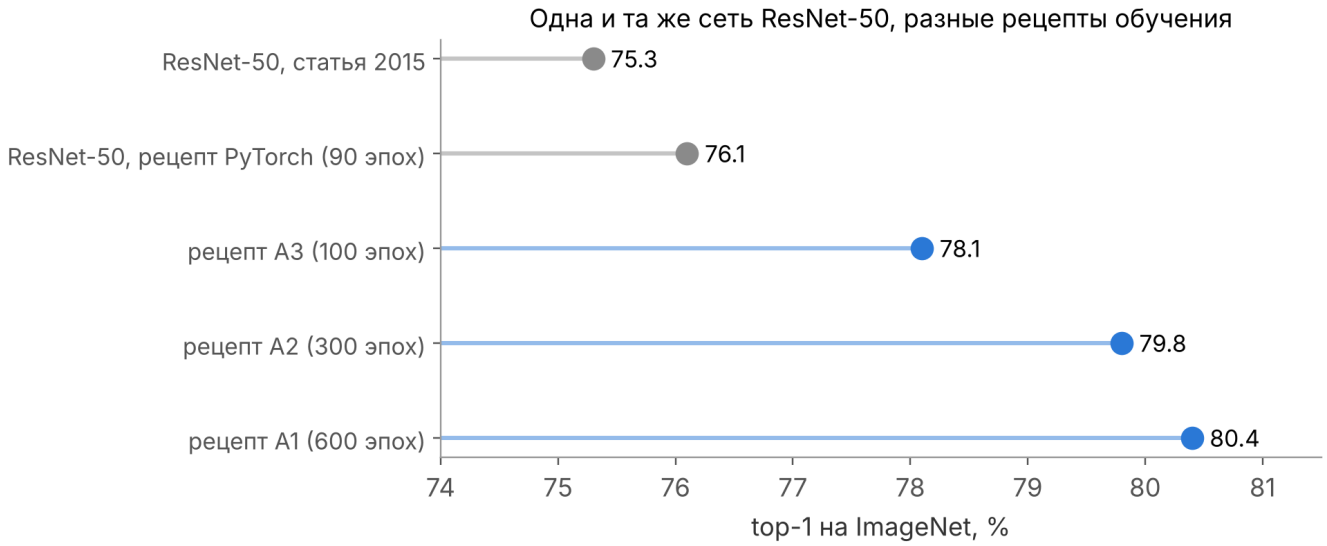
Автоматический подбор политики

- AutoAugment (2018): политику (какие операции, с какой вероятностью и силой) ищут обучением с подкреплением; тысячи GPU-часов на поиск
  - политика подбирается под датасет (поиск идёт на его уменьшенной копии с маленькой сетью)
  - сила — насколько существенно аугментация меняет картинку (градус поворота, степень изменения яркости и т.д.), в единой шкале
- RandAugment (2019): всего два гиперпараметра — число операций N и общая сила M; операции берутся случайно. На ImageNet ResNet-50: 76.3 → 77.6, столько же, сколько у AutoAugment
- TrivialAugment (2021): одна случайная операция со случайной силой, гиперпараметров нет — не хуже
- Вывод: раз выученная политика всё равно случайна и не зависит от картинки, простой случайный выбор преобразований работает не хуже

https://arxiv.org/abs/1909.13719 · https://arxiv.org/abs/2103.10158

Насколько это важно

- Рецепт обучения — оптимизатор и его гиперпараметры, расписание шага, число эпох, размер батча, аугментации, регуляризация, функция потерь, разрешение картинок, …
- Та же сеть ResNet-50, другой рецепт приводят к росту с 75.3% до 80.4% top-1 на ImageNet
- Сильные аугментации требуют долгого обучения, лучший рецепт — 600 эпох вместо 90
- Рецепт часто даёт больше, чем новая архитектура — сравнивать архитектуры честно можно только при одинаковом рецепте



Общее для A1–A3: LAMB + cosine, батч 2048, RandAugment, Mixup, CutMix, BCE-лосс
Отличия: 600 / 300 / 100 эпох; сила RandAugment 7 / 7 / 6; Mixup α 0.2 / 0.1 / 0.1; разрешение 224 / 224 / 160; repeated augmentation и stochastic depth — в A1 и A2; label smoothing — только в A1

https://arxiv.org/abs/2110.00476

Оговорки и варианты

- Преобразование не должно менять метку: отражение цифр 6 и 9, поворот текста — нельзя; для спутниковых и медицинских снимков повороты, наоборот, естественны
- Test-time augmentation: на тесте несколько аугментаций одной картинки, предсказания усредняются (стандартный приём на соревнованиях)
- Другие модальности: текст — обратный перевод и замена синонимов; аудио — SpecAugment, маскирование полос спектрограммы
- Аугментация лишь добавляет инвариантности и не заменяет обогащение выборки